CSE-207

Dynamic Programming

All Pairs Shortest Paths and Floyd-Warshall Algorithm

All-pairs shortest paths

- Given a weighted directed graph, *G(V, E)* with a weight function *w* that maps edges to real-valued weights.
  - *w(u,v)* denote the weight of an edge *(u,v)*

- For every pair of vertices *u, v*, find a shortest path from *u* to *v*, where the weight of a path is the sum of the weights of the edges along the path.

All-pairs shortest paths

- If we allow negative edge weights but no negative-weight cycles
  - Cannot use Dijkstra' algorithm

All-pairs shortest paths

- Instead, we will use a dynamic programming solution:
  - Floyd-Warshall Algorithm
  - Running time: $O(V^3)$

- Let $\delta(u, v)$ denote the weight of the shortest path from *u* to *v*

Input: adjacency matrix representing G

- Assume an adjacency matrix representation
  - Assume vertices are numbered 1,2,…,n
  - The input is a *n x n* matrix $W = (w_{ij})$ representing the edge weights of an n-vertex directed graph

$$w_{ij} = \begin{cases} 0 & \text{if } i = j \\ w(i,j) & \text{if } i \neq j \text{ and } (i,j) \in E \\ INF & \text{if } i \neq j \text{ and } (i,j) \notin E \end{cases}$$

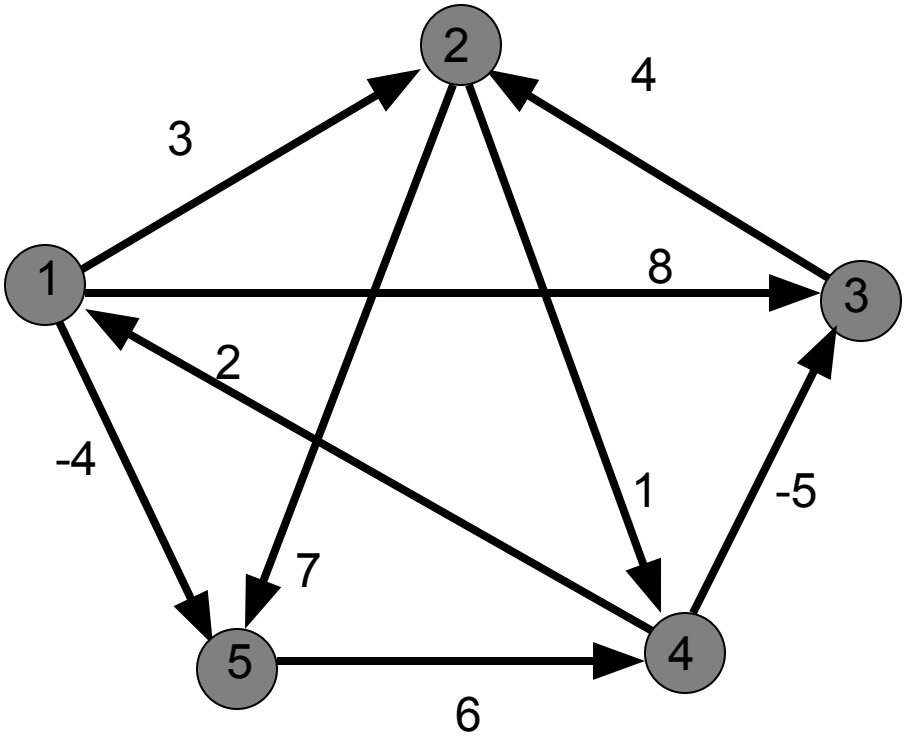
Output?

- *n x n* matrix $D = (d_{ij})$

- Entry d_{ij} will contain the weight of the shortest path from vertex *i* to vertex *j*, that is

$$d_{ij} = \delta(i, j)$$

Example: input



W

	1	2	3	4	5
1	0	3	8	∞	-4
2	∞	0	∞	1	7
3	∞	4	0	∞	∞
4	2	∞	-5	0	∞
5	∞	∞	∞	6	0

DP Formulation

- Intermediate vertices: on a path $p = < v_1, v_2, ..., v_l >$ any vertex other than the first and the last vertices is called and intermediate vertex.

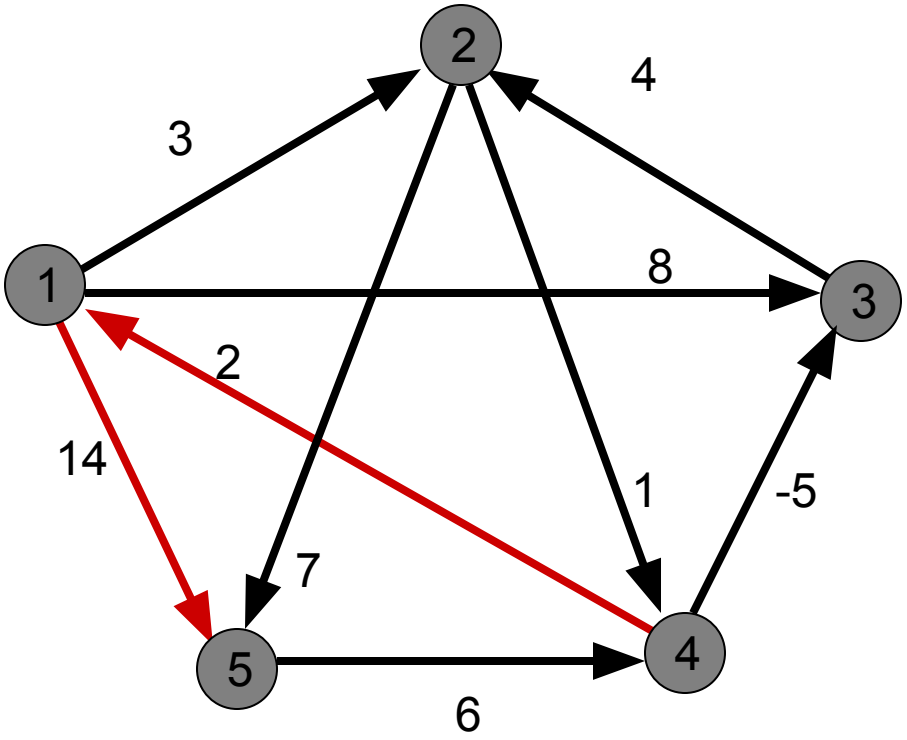
- Let vertices of G be V={1,2,….,n}
- Consider a subset {1,2,…,k} of these vertices for some k
- Subproblems: limit the set of intermediate vertices on a path from i to j to subset {1,2,…,k}
  - The path is allowed to pass through only vertices 1 through k

DP Formulation

- In the original problem: For any pair of vertices *i* and *j*, the intermediate vertices could be drawn from the set {1,…,n}

- Define $d_{ij}^{(k)}$ to be the shortest path from i to j such that intermediate vertices on the path are drawn from the set {1,2,…,k}
  - Matrix $D^{(k)}$

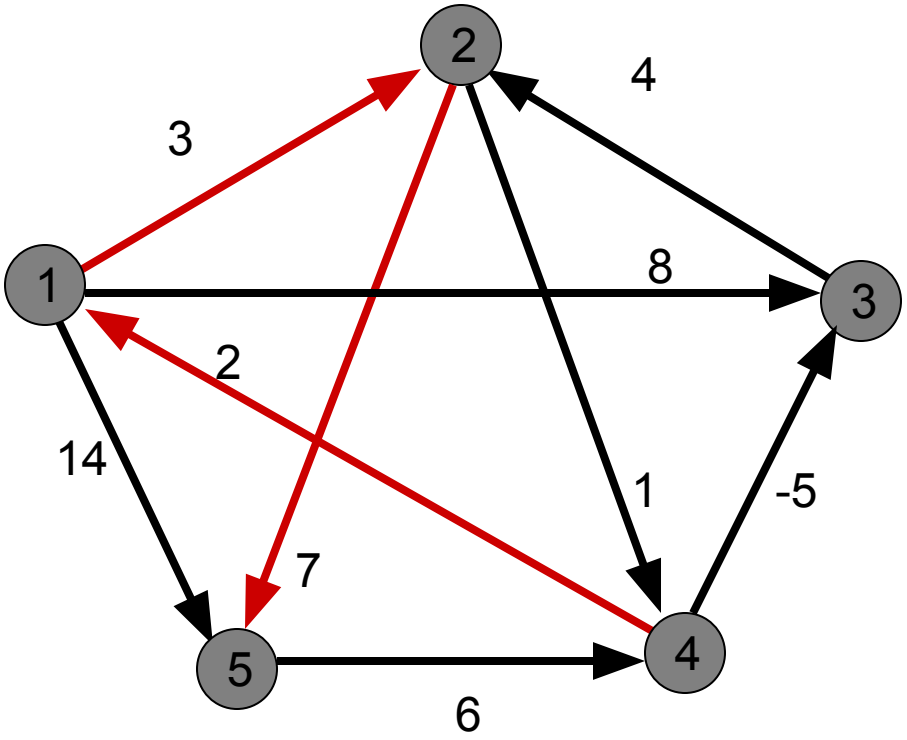
Example



$$d_{45}^{(0)} = \infty$$

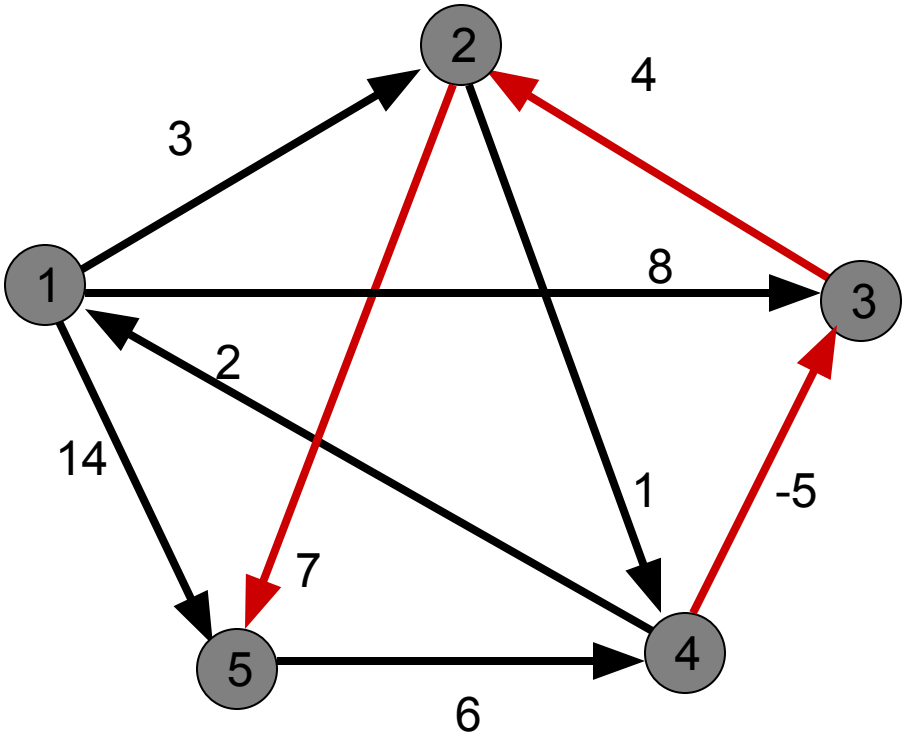
$$d_{45}^{(1)} = 16$$

Example



$$d_{45}^{(2)} = 12$$

Example



$$d_{45}^{(3)} = 6$$

DP Formulation

- How to reduce $d_{ij}^{(k)}$ to smaller problems? i.e. how to compute $d_{ij}^{(k)}$ assuming we have already computed $D^{(k-1)}$?

DP Formulation

- How to reduce $d_{ij}^{(k)}$ to smaller problems? i.e. how to compute $d_{ij}^{(k)}$ assuming we have already computed $D^{(k-1)}$?

- Two cases:

  Case 1: Vertex k is NOT among the intermediate vertices on the shortest path from i to j

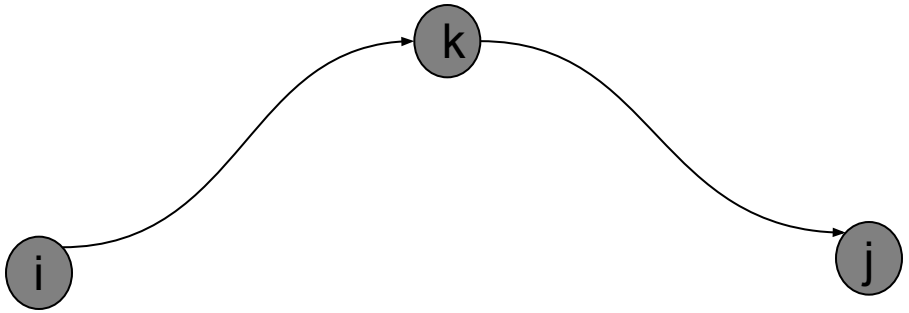
$$d_{ij}^{(k)} = d_{ij}^{(k-1)}$$

DP Formulation

Case 2: Vertex k is an intermediate vertex on the shortest path from i to j

- First take the shortest path from i to k using intermediate vertices from the set {1,2,…,k-1}
- Then take the shortest path from k to j using intermediate vertices from the set {1,2,…,k-1}

$$d_{ij}^{(k)} = d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$$



DP Formulation

- Base case:

$$d_{ij}^{(0)} = w_{ij}$$

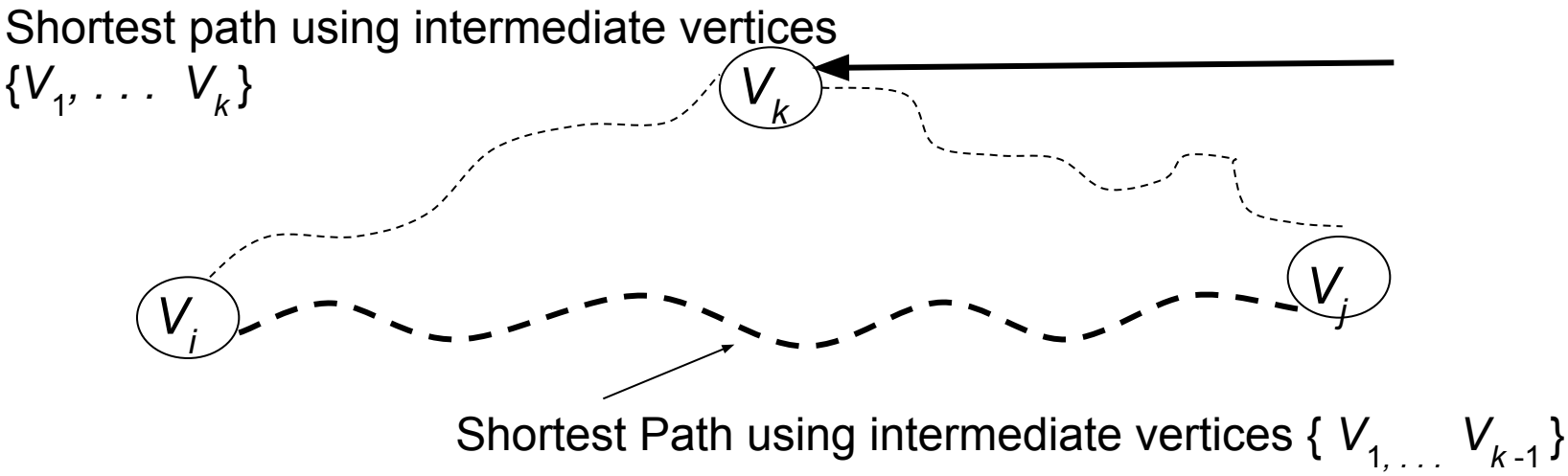
- Recursive definition (for *k* > *0*)

$$d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$$

The Recursive Definition:

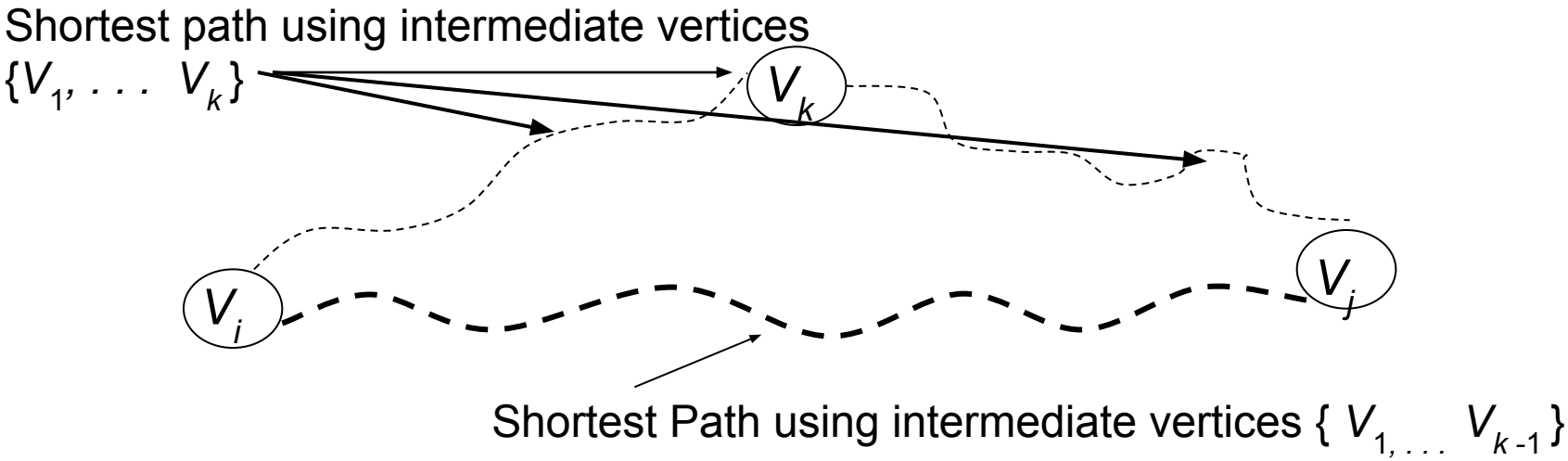
Case 1: A shortest path from v_i to v_j restricted to using only vertices from $\{v_1, v_2, \ldots, v_k\}$ as intermediate vertices does not use v_k. Then $D^{(k)}[i,j] = D^{(k-1)}[i,j]$.

Case 2: A shortest path from v_i to v_j restricted to using only vertices from $\{v_1, v_2, \ldots, v_k\}$ as intermediate vertices does use v_k. Then $D^{(k)}[i,j] = D^{(k-1)}[i,k] + D^{(k-1)}[k,j]$.



The recursive definition

- **Since**
  $D^{(k)}[i,j] = D^{(k-1)}[i,j]$ **or**
  $D^{(k)}[i,j] = D^{(k-1)}[i,k] + D^{(k-1)}[k,j]$.
  We conclude:
  $D^{(k)}[i,j] = \min\{ D^{(k-1)}[i,j], D^{(k-1)}[i,k] + D^{(k-1)}[k,j] \}$.



Floyd-Warshall Algorithm

Floyd-Warshall(*W*) {

n = rows[*W*]

$D^{(0)} = W$

for k=1 to n do

for i=1 to n do

for j=1 to n do

$$d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$$

return $D^{(n)}$

}

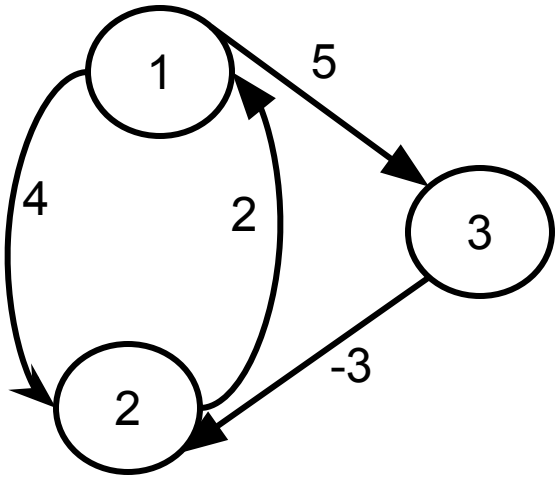
$O(n^3)$

Floyd's Algorithm

Floyd//Computes shortest distance between all pairs of
//nodes, and saves P to enable finding shortest paths

1. $D^0 \leftarrow W$ // initialize *D* array to *W* [ ]
2. $P \leftarrow 0$ // initialize P array to [0]
3. **for** $k \leftarrow 1$ **to** n
4. **do for** $i \leftarrow 1$ **to** n
5. **do for** $j \leftarrow 1$ **to** n
6. **if** $(D^{k-1}[\, i, j\,] > D^{k-1}[\, i, k\,] + D^{k-1}[\, k, j\,])$
7. **then** $D^k[\, i, j\,] \leftarrow D^{k-1}[\, i, k\,] + D^{k-1}[\, k, j\,]$
8. $P[\, i, j\,] \leftarrow k$;
9. **else** $D^k[\, i, j\,] \leftarrow D^{k-1}[\, i, j\,]$

Example

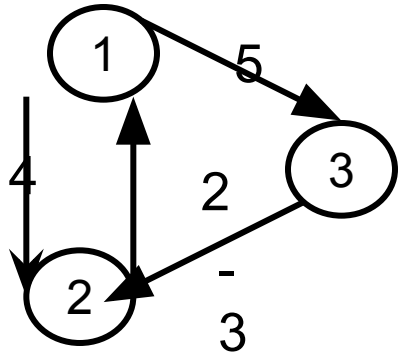


$W = D^0 =$

	1	2	3
1	0	4	5
2	2	0	∞
3	∞	-3	0

$P =$

	1	2	3
1	0	0	0
2	0	0	0
3	0	0	0



D^0 =

	1	2	3
1	0	4	5
2	2	0	∞
3	∞	-3	0

k = 1
Vertex 1 can be intermediate node

D^1 =

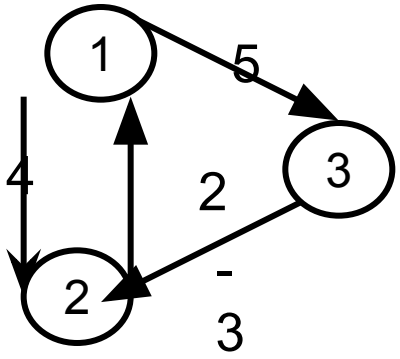
	1	2	3
1	0	4	5
2	2	0	7
3	∞	-3	0

$D^1[2,3] = \min(D^0[2,3], D^0[2,1]+D^0[1,3])$
$= \min (\infty, 7)$
$= 7$

P =

	1	2	3
1	0	0	0
2	0	0	1
3	0	0	0

$D^1[3,2] = \min(D^0[3,2], D^0[3,1]+D^0[1,2])$
$= \min (-3, \infty)$
$= -3$



D^1 =

	1	2	3
1	0	4	5
2	2	0	7
3	∞	-3	0

k = 2
Vertices 1, 2 can be intermediate

D^2 =

	1	2	3
1	0	4	5
2	2	0	7
3	-1	-3	0

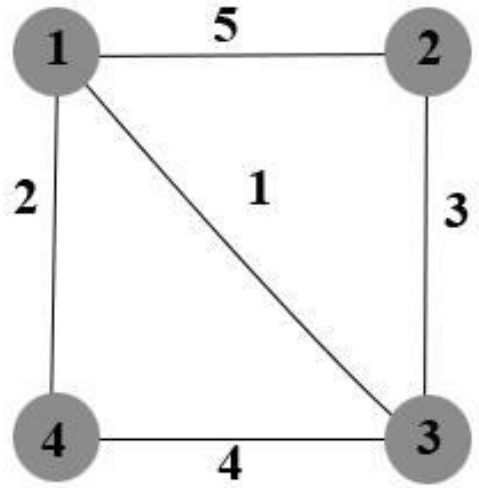
$D^2[1,3] = \min(D^1[1,3], D^1[1,2]+D^1[2,3])$
$= \min (5, 4+7)$
$= 5$

P =

	1	2	3
1	0	0	0
2	0	0	1
3	2	0	0

$D^2[3,1] = \min(D^1[3,1], D^1[3,2]+D^1[2,1])$
$= \min (\infty, -3+2)$
$= -1$

Floyd-Warshall Algorithm (continue)



D_0

	1	2	3	4
1	0	5	1	2
2	5	0	3	∞
3	1	3	0	4
4	2	∞	4	0

S_0

	1	2	3	4
1	0	2	3	4
2	1	0	3	4
3	1	2	0	4
4	1	2	3	0

Floyd-Warshall Algorithm (continue)

If $d_{ij} > d_{ik} + d_{kj}$

In D_k

$C_{ij}=d_{ik} + d_{kj}$

Else

In D_k

$C_{ij}=d_{ij}$ of D_{k-1}

D_0

	1	2	3	4
1	0	5	1	2
2	5	0	3	∞
3	1	3	0	4
4	2	∞	4	0

S_0

	1	2	3	4
1	0	2	3	4
2	1	0	3	4
3	1	2	0	4
4	1	2	3	0

Iteration 1 , k=1

$C_{23} = d_{23} > d_{21} + d_{13}$

$=3>5+1$

$=3$

$C_{24} = d_{24} > d_{21} + d_{14}$

$= \infty > 5+2$

$=7$

$C_{32} = d_{32} > d_{31} + d_{12}$

$= 3 > 1+5$

$= 3$

$C_{34} = d_{34} > d_{31} + d_{14}$

$= 4 > 1+2$

$= 3$

$C_{42} = d_{42} > d_{41} + d_{12}$

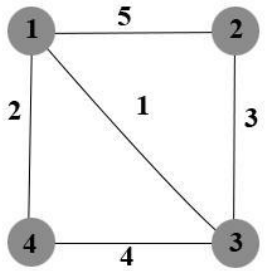
$= \infty > 2+5$

$=7$

$C_{43} = d_{43} > d_{41} + d_{13}$

$= 3 > 2+1$

$=3$



D_1

	1	2	3	4
1	0	5	1	2
2	5	0	3	7
3	1	3	0	3
4	2	7	3	0

S_1

	1	2	3	4
1	0	2	3	4
2	1	0	3	1
3	1	2	0	1
4	1	1	1	0

i=row

J=column

Floyd-Warshall Algorithm (continue)

Iteration 2 , k=2

$C_{13} = d_{13} > d_{12} + d_{23}$

$=1>5+3$

$=1$

$C_{14} = d_{14} > d_{12} + d_{24}$

$= 2 >5+7$

$=2$

$C_{31} = d_{31} > d_{32} + d_{21}$

$= 1 >3+5$

$= 1$

$C_{34} = d_{34} > d_{32} + d_{24}$

$= 3 >3+7$

$= 3$

$C_{41} = d_{41} > d_{42} + d_{21}$

$= 2 >7+5$

$=2$

$C_{43} = d_{43} > d_{42} + d_{23}$

$= 3 >7+3$

$=3$

D_1

	1	2	3	4
1	0	5	1	2
2	5	0	3	7
3	1	3	0	3
4	2	7	3	0

S_1

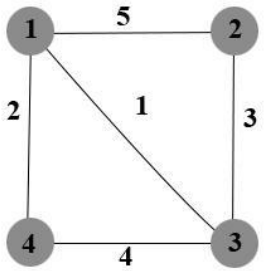
	1	2	3	4
1	0	2	3	4
2	1	0	3	1
3	1	2	0	1
4	1	1	1	0

D_2

	1	2	3	4
1	0	5	1	2
2	5	0	3	7
3	1	3	0	3
4	2	7	3	0

S_2

	1	2	3	4
1	0	2	3	4
2	1	0	3	1
3	1	2	0	1
4	1	1	1	0



Floyd-Warshall Algorithm (continue)

Iteration 3 , k=3

$C_{12} = d_{12} > d_{13} + d_{32}$

=5>1+3

=4

$C_{14} = d_{14} > d_{13} + d_{34}$

= 2 >1+3

=2

$C_{21} = d_{21} > d_{23} + d_{31}$

= 5 >3+1

= 4

$C_{24} = d_{24} > d_{23} + d_{34}$

= 7 >3+3

= 6

$C_{41} = d_{41} > d_{43} + d_{32}$

= 2 >3+1

=2

$C_{42} = d_{42} > d_{43} + d_{32}$

= 7 >3+3

=6

D_2

	1	2	3	4
1	0	5	1	2
2	5	0	3	7
3	1	3	0	3
4	2	7	3	0

S_2

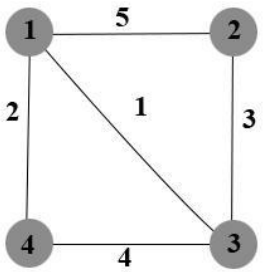
	1	2	3	4
1	0	2	3	4
2	1	0	3	1
3	1	2	0	1
4	1	1	1	0

D_3

	1	2	3	4
1	0	4	1	2
2	4	0	3	6
3	1	3	0	3
4	2	6	3	0

S_3

	1	2	3	4
1	0	3	3	4
2	3	0	3	3
3	1	2	0	1
4	1	3	1	0



Floyd-Warshall Algorithm (continue)

Iteration 4 , k=4

$C_{12} = d_{12} > d_{14} + d_{42}$

$=4>2+2$

$=4$

$C_{13} = d_{13} > d_{14} + d_{43}$

$= 1 > 2+3$

$=1$

$C_{21} = d_{21} > d_{24} + d_{41}$

$= 4 > 6+2$

$= 4$

$C_{23} = d_{23} > d_{23} + d_{33}$

$= 3 > 3+0$

$= 3$

$C_{31} = d_{31} > d_{33} + d_{31}$

$= 1 > 0+1$

$=1$

$C_{32} = d_{32} > d_{33} + d_{32}$

$= 3 > 0+3$

$=3$

D_3

	1	2	3	4
1	0	4	1	2
2	4	0	3	6
3	1	3	0	3
4	2	6	3	0

S_3

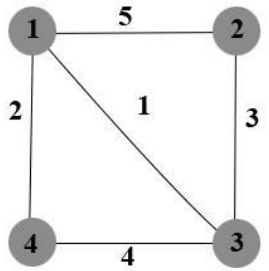
	1	2	3	4
1	0	3	3	4
2	3	0	3	3
3	1	2	0	1
4	1	3	1	0

D_4

	1	2	3	4
1	0	4	1	2
2	4	0	3	6
3	1	3	0	3
4	2	6	3	0

S_4

	1	2	3	4
1	0	3	3	4
2	3	0	3	3
3	1	2	0	1
4	1	3	1	0



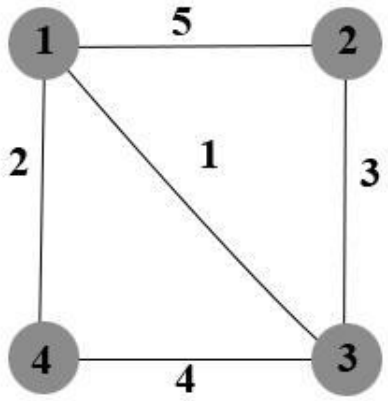
Floyd-Warshall Algorithm (continue)

D_4

	1	2	3	4
1	0	4	1	2
2	4	0	3	6
3	1	3	0	3
4	2	6	3	0

S_4

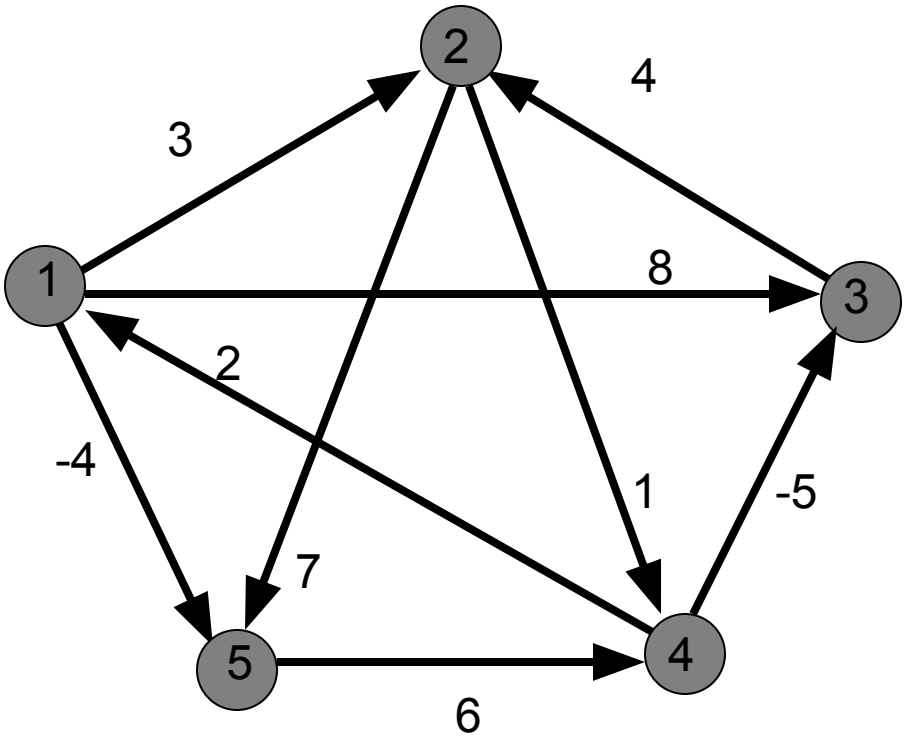
	1	2	3	4
1	0	3	3	4
2	3	0	3	3
3	1	2	0	1
4	1	3	1	0



The required shortest path from all vertices

Source vertex (i)	Destination vertex (j)	Distance	Shortest path
1	2	4	1->3->2
1	3	1	1->3
1	4	2	1->4
2	3	3	2->3
2	4	6	2->3->1->4
3	4	3	3->1->4

Example

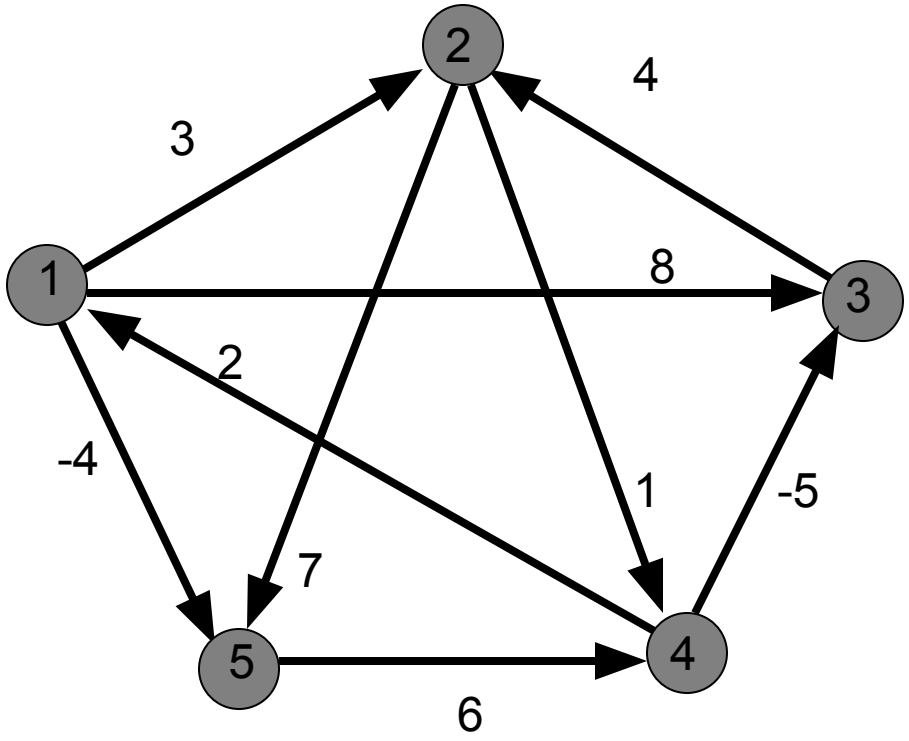


$$D^{(0)} = W$$

	1	2	3	4	5
1	0	3	8	∞	-4
2	∞	0	∞	1	7
3	∞	4	0	∞	∞
4	2	∞	-5	0	∞
5	∞	∞	∞	6	0

Example

	1	2	3	4	5
1	0	3	8	∞	-4
2	∞	0	∞	1	7
3	∞	4	0	∞	∞
4	2	∞	-5	0	∞
5	∞	∞	∞	6	0

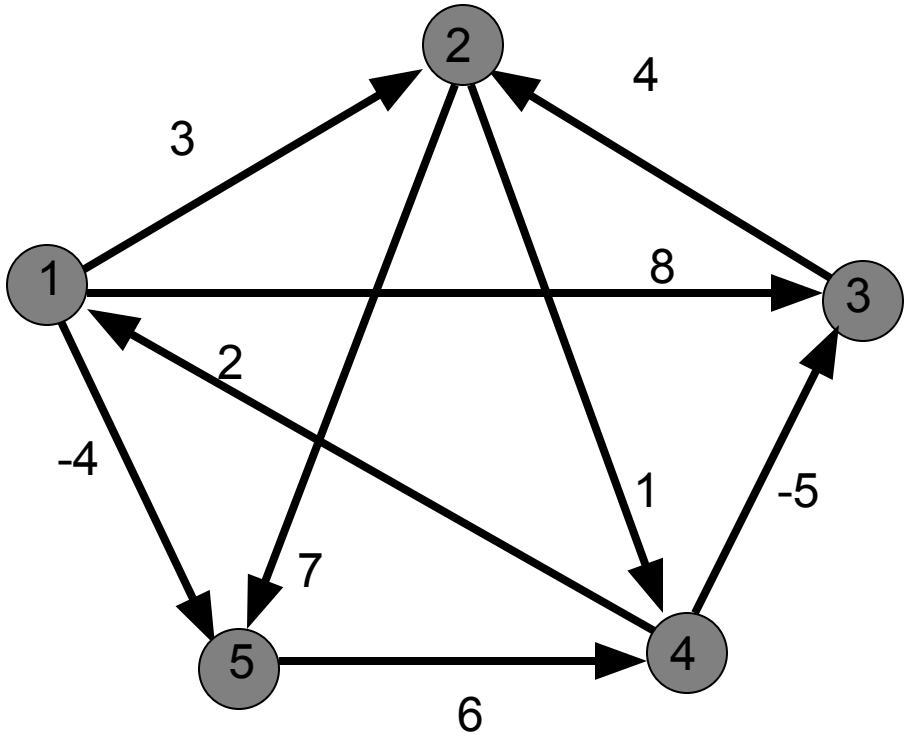


$D^{(1)}$

	1	2	3	4	5
1	0	3	8	∞	-4
2	∞	0	∞	1	7
3	∞	4	0	∞	∞
4	2	5	-5	0	-2
5	∞	∞	∞	6	0

Example

	1	2	3	4	5
1	0	3	8	∞	-4
2	∞	0	∞	1	7
3	∞	4	0	∞	∞
4	2	5	-5	0	-2
5	∞	∞	∞	6	0

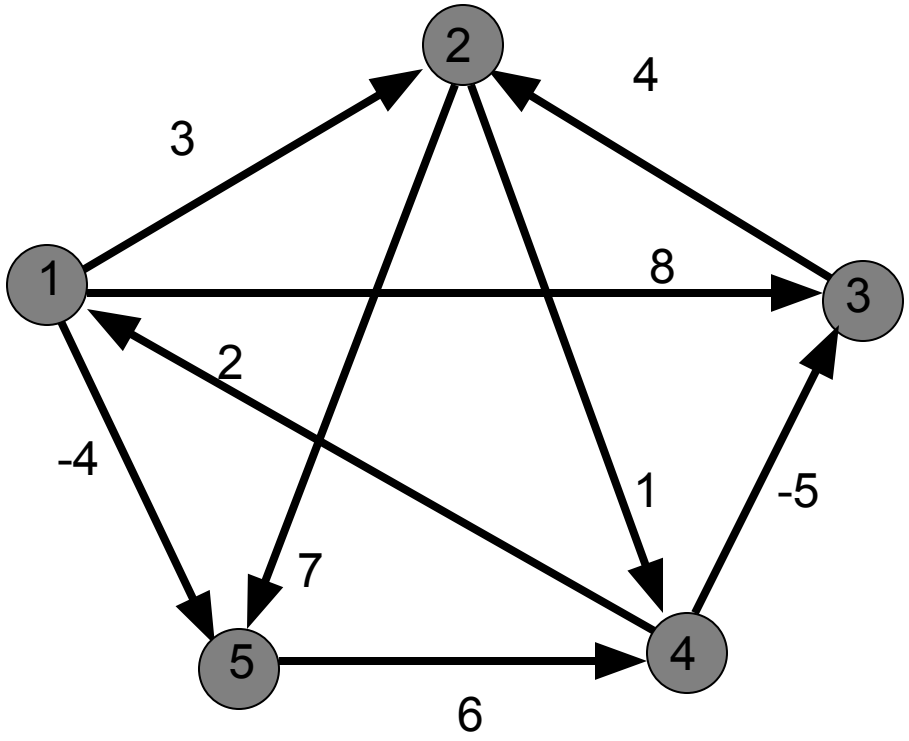


$D^{(2)}$

	1	2	3	4	5
1	0	3	8	4	-4
2	∞	0	∞	1	7
3	∞	4	0	5	11
4	2	5	-5	0	-2
5	∞	∞	∞	6	0

Example

	1	2	3	4	5
1	0	3	8	4	-4
2	∞	0	∞	1	7
3	∞	4	0	5	11
4	2	5	-5	0	-2
5	∞	∞	∞	6	0

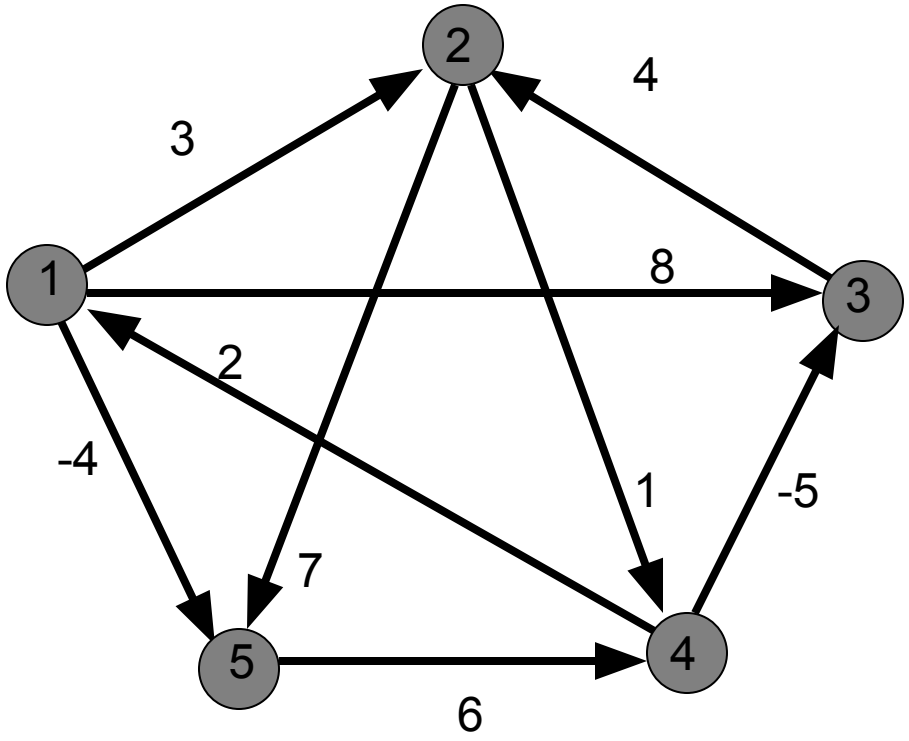


$D^{(3)}$

	1	2	3	4	5
1	0	3	8	4	-4
2	∞	0	∞	1	7
3	∞	4	0	5	11
4	2	-1	-5	0	-2
5	∞	∞	∞	6	0

Example

	1	2	3	4	5
1	0	3	8	4	-4
2	∞	0	∞	1	7
3	∞	4	0	5	11
4	2	-1	-5	0	-2
5	∞	∞	∞	6	0

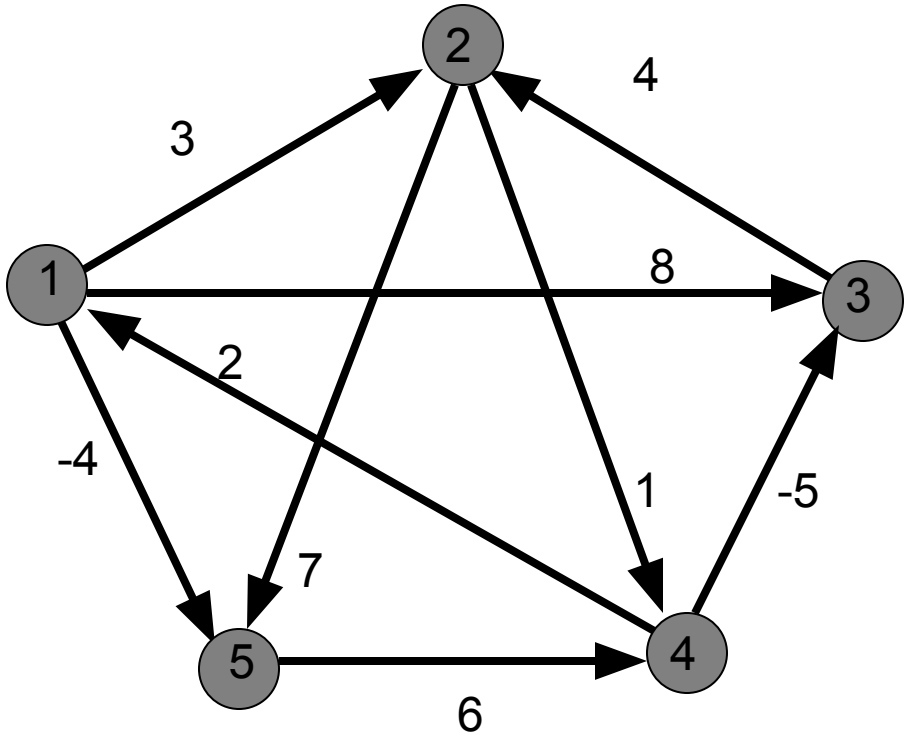


$D^{(4)}$

	1	2	3	4	5
1	0	3	-1	4	-4
2	3	0	-4	1	-1
3	7	4	0	5	3
4	2	-1	-5	0	-2
5	8	5	1	6	0

Example

	1	2	3	4	5
1	0	3	-1	4	-4
2	3	0	-4	1	-1
3	7	4	0	5	3
4	2	-1	-5	0	-2
5	8	5	1	6	0



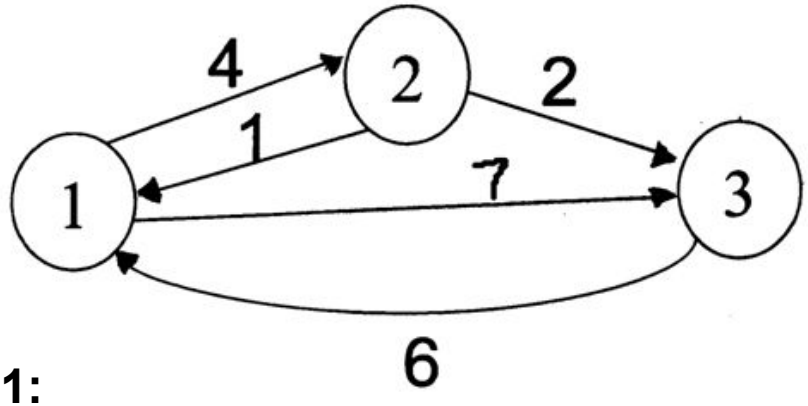
$D^{(5)}$

	1	2	3	4	5
1	0	1	-3	2	-4
2	3	0	-4	1	-1
3	7	4	0	5	3
4	2	-1	-5	0	-2
5	8	5	1	6	0

Floyd Warshall Algorithm - Example

$$D^{(0)} = \begin{bmatrix} 0 & 4 & 7 \\ 1 & 0 & 2 \\ 6 & \infty & 0 \end{bmatrix}$$

Original weights.



$$D^{(1)} = \begin{bmatrix} 0 & 4 & 7 \\ 1 & 0 & 2 \\ 6 & 10 & 0 \end{bmatrix}$$

Consider Vertex 1:

D(3,2) = D(3,1) + D(1,2)

$$D^{(2)} = \begin{bmatrix} 0 & 4 & 6 \\ 1 & 0 & 2 \\ 6 & 10 & 0 \end{bmatrix}$$

Consider Vertex 2:

D(1,3) = D(1,2) + D(2,3)

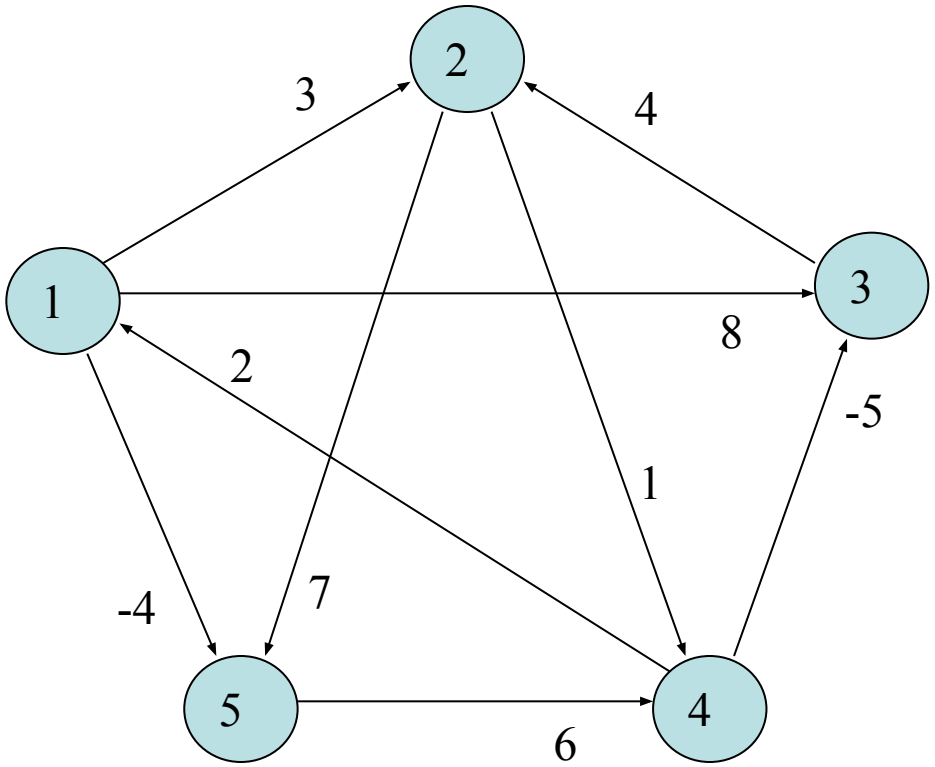
$$D^{(3)} = \begin{bmatrix} 0 & 4 & 6 \\ 1 & 0 & 2 \\ 6 & 10 & 0 \end{bmatrix}$$

Consider Vertex 3:

Nothing changes.

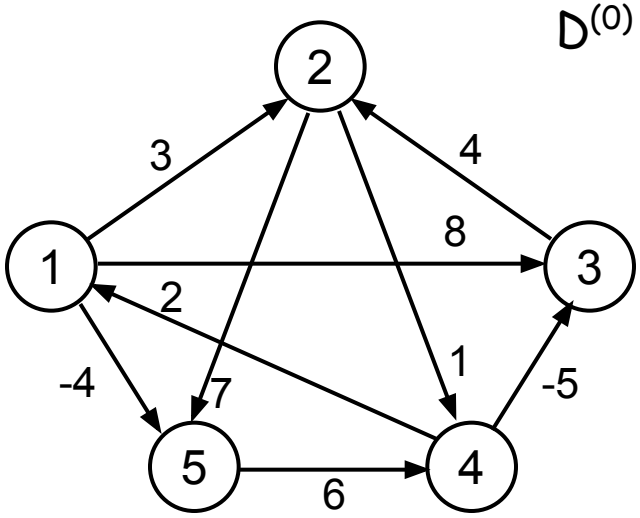
Shortest Paths and Matrix Multiplication

- Example



Example

$$d_{ij}^{(k)} = \min \{ d_{ij}^{(k-1)} , d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \}$$



$D^{(0)} = W$

	1	2	3	4	5
1	0	3	8	∞	-4
2	∞	0	∞	1	7
3	∞	4	0	∞	∞
4	2	∞	-5	0	∞
5	∞	∞	∞	6	0

$D^{(1)}$

	1	2	3	4	5
1	0	3	8	∞	-4
2	∞	0	∞	1	7
3	∞	4	0	∞	∞
4	2	5	-5	0	-2
5	∞	∞	∞	6	0

$D^{(2)}$

	1	2	3	4	5
1	0	3	8	4	-4
2	∞	0	∞	1	7
3	∞	4	0	5	11
4	2	5	-5	0	-2
5	∞	∞	∞	6	0

$D^{(3)}$

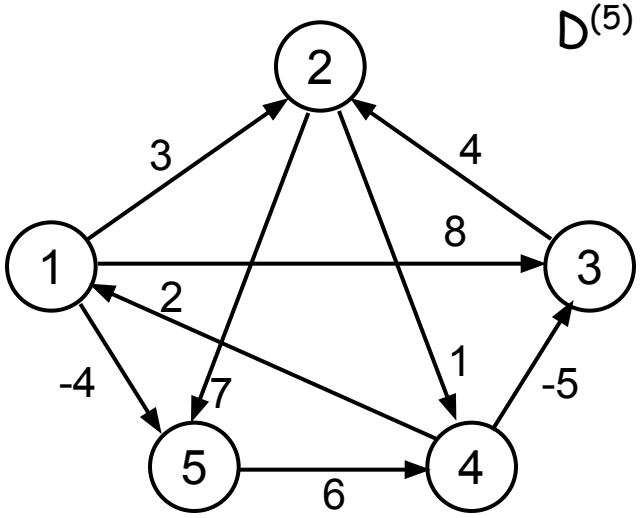
0	3	8	4	-4
∞	0	∞	1	7
∞	4	0	5	11
2	-1	-5	0	-2
∞	∞	∞	6	0

$D^{(4)}$

0	3	-1	4	-4
3	0	-4	1	-1
7	4	0	5	3
2	-1	-5	0	-2
8	5	1	6	0

Example

$$d_{ij}^{(k)} = \min \{ d_{ij}^{(k-1)} , d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \}$$



$D^{(5)}$

	1	2	3	4	5
1	0	1	-3	2	-4
2	3	0	-4	1	-1
3	7	4	0	5	3
4	2	-1	-5	0	-2
5	8	5	1	6	0

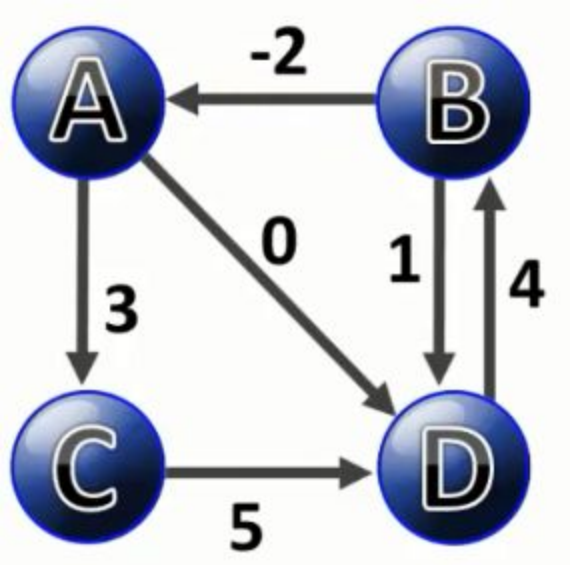
$P^{(5)}$

	1	2	3	4	5
1	-	3	4	5	1
2	4	-	4	2	1
3	4	3	-	2	1
4	4	3	4	-	1
5	4	3	4	5	-

Source: 5, Destination: 1
Shortest path: 8
Path: 5 …1 : 5…4…1: 5->4…1: 5->4->1

Source: 1, Destination: 3
Shortest path: -3
Path: 1 …3 : 1…4…3: 1…5…4…3: 1->5->4->3

Example



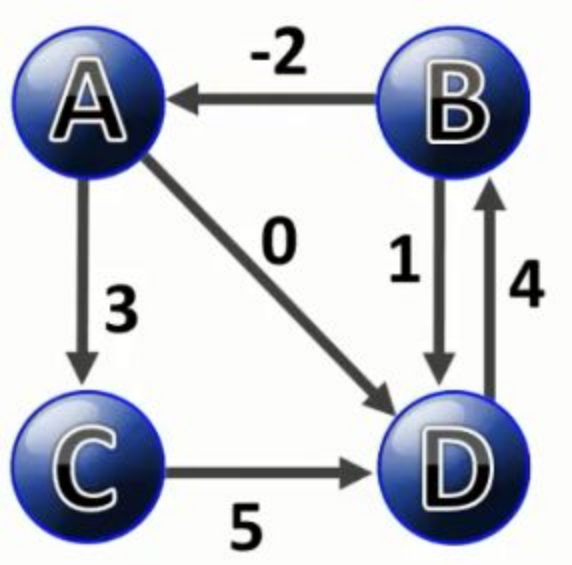
to

from / d0	A	B	C	D
A	0	∞	3	0
B	-2	0	∞	1
C	∞	∞	0	5
D	∞	4	∞	0

π0	A	B	C	D
A	-	-	A	-
B	B	-	-	B
C	-	-	-	C
D	-	D	-	-

Example

- We update using A as stopping node



from \ to

d0	A	B	C	D
A	0	∞	3	0
B	-2	0	∞	1
C	∞	∞	0	5
D	∞	4	∞	0

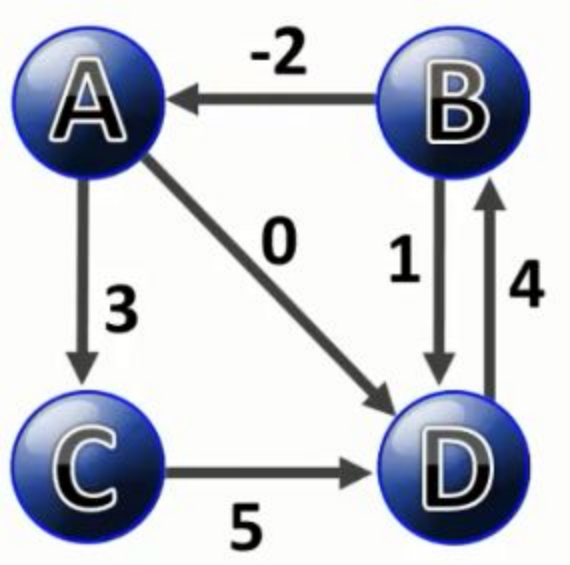
π0	A	B	C	D
A	-	-	A	-
B	B	-	-	B
C	-	-	-	C
D	-	D	-	-

d1	A	B	C	D
A	0	∞	3	0
B	-2	0	∞	1
C	∞	∞	0	5
D	∞	4	∞	0

π1	A	B	C	D
A	-	-	A	-
B	B	-	-	B
C	-	-	-	C
D	-	D	-	-

AA + AA < AA ? NO.

Example



to

from d0	A	B	C	D
A	0	∞	3	0
B	-2	0	∞	1
C	∞	∞	0	5
D	∞	4	∞	0

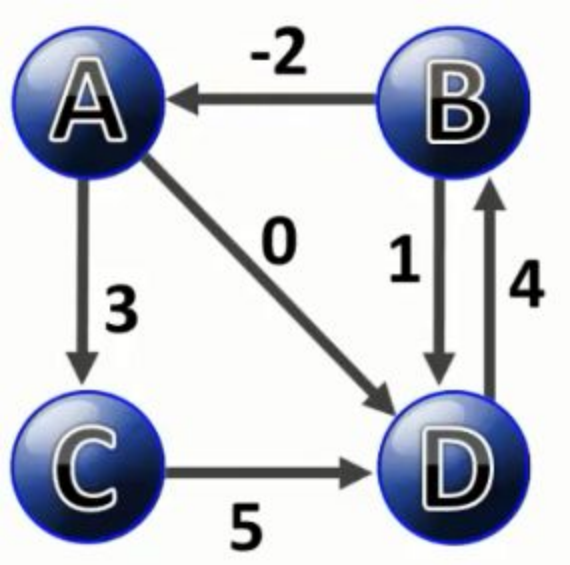
π0	A	B	C	D
A	-	-	A	-
B	B	-	-	B
C	-	-	-	C
D	-	D	-	-

d1	A	B	C	D
A	0	∞	3	0
B	-2	0	∞	1
C	∞	∞	0	5
D	∞	4	∞	0

π1	A	B	C	D
A	-	-	A	-
B	B	-	-	B
C	-	-	-	C
D	-	D	-	-

AA + AB < AB ? NO.

Example



from \ to

d0	A	B	C	D
A	0	∞	3	0
B	-2	0	∞	1
C	∞	∞	0	5
D	∞	4	∞	0

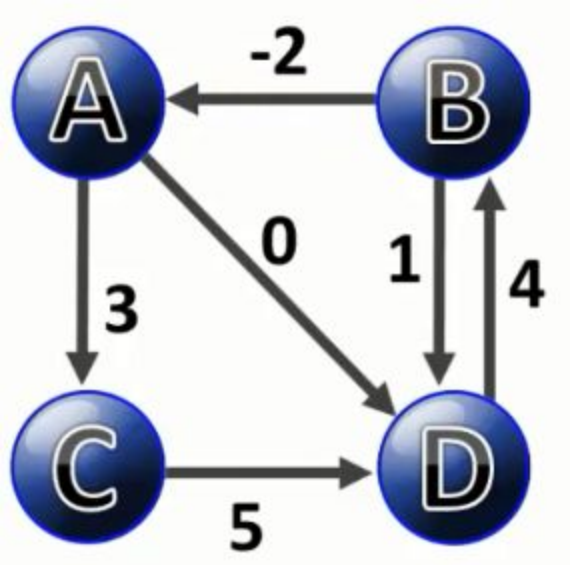
π0	A	B	C	D
A	-	-	A	-
B	B	-	-	B
C	-	-	-	C
D	-	D	-	-

d1	A	B	C	D
A	0	∞	3	0
B	-2	0	∞	1
C	∞	∞	0	5
D	∞	4	∞	0

π1	A	B	C	D
A	-	-	A	-
B	B	-	-	B
C	-	-	-	C
D	-	D	-	-

AA + AC < AC ? NO.

Example



to

from

d0	A	B	C	D
A	0	∞	3	0
B	-2	0	∞	1
C	∞	∞	0	5
D	∞	4	∞	0

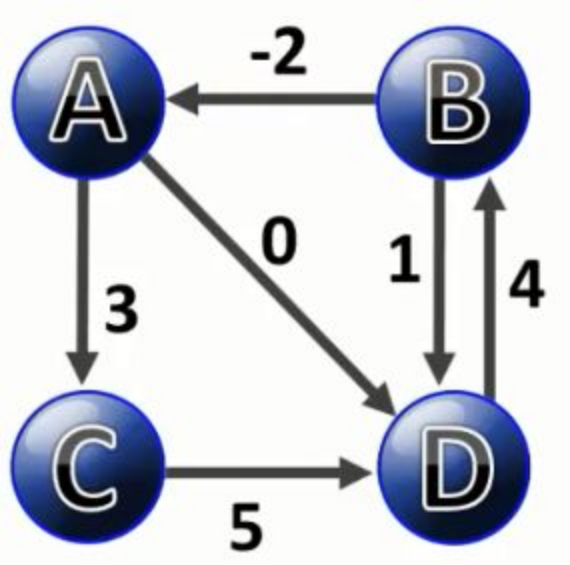
π0	A	B	C	D
A	-	-	A	-
B	B	-	-	B
C	-	-	-	C
D	-	D	-	-

d1	A	B	C	D
A	0	∞	3	0
B	-2	0	∞	1
C	∞	∞	0	5
D	∞	4	∞	0

π1	A	B	C	D
A	-	-	A	-
B	B	-	-	B
C	-	-	-	C
D	-	D	-	-

BA + AC < BC ? YES! BA + AC = 1

Example



to

from

d0	A	B	C	D
A	0	∞	3	0
B	-2	0	∞	1
C	∞	∞	0	5
D	∞	4	∞	0

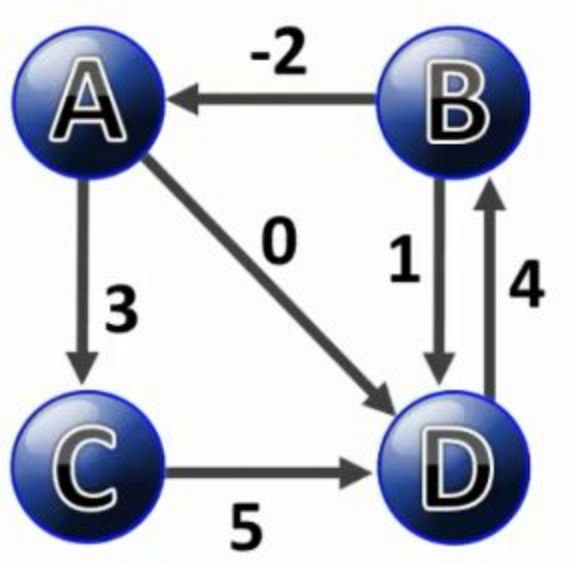
π0	A	B	C	D
A	-	-	A	-
B	B	-	-	B
C	-	-	-	C
D	-	D	-	-

d1	A	B	C	D
A	0	∞	3	0
B	-2	0	1	1
C	∞	∞	0	5
D	∞	4	∞	0

π1	A	B	C	D
A	-	-	A	-
B	B	-	A	B
C	-	-	-	C
D	-	D	-	-

BA + AD < BD ? YES! BA + AD = -2

Example



to

from

d0	A	B	C	D
A	0	∞	3	0
B	-2	0	∞	1
C	∞	∞	0	5
D	∞	4	∞	0

π0	A	B	C	D
A	-	-	A	-
B	B	-	-	B
C	-	-	-	C
D	-	D	-	-

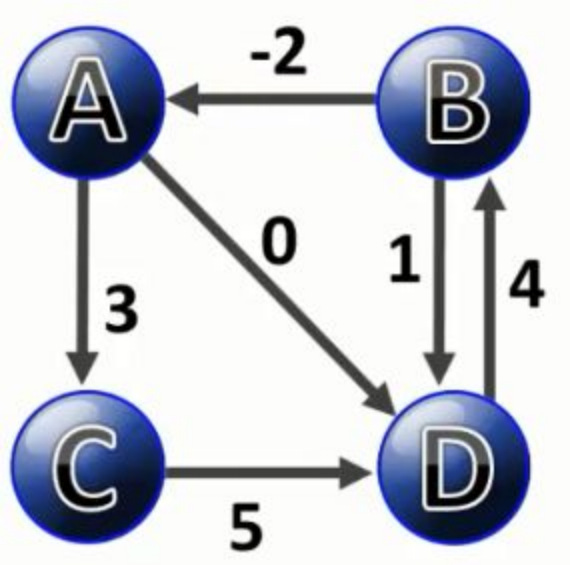
d1	A	B	C	D
A	0	∞	3	0
B	-2	0	1	1
C	∞	∞	0	5
D	∞	4	∞	0

π1	A	B	C	D
A	-	-	A	-
B	B	-	A	B
C	-	-	-	C
D	-	D	-	-

BA + AD < BD ? YES! BA + AD = -2

Example

- Complete the first iteration



to / from

d0	A	B	C	D
A	0	∞	3	0
B	-2	0	∞	1
C	∞	∞	0	5
D	∞	4	∞	0

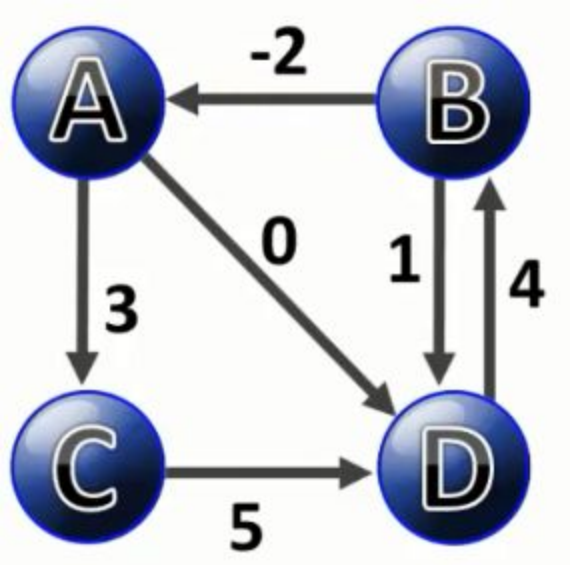
π0	A	B	C	D
A	-	-	A	-
B	B	-	-	B
C	-	-	-	C
D	-	D	-	-

d1	A	B	C	D
A	0	∞	3	0
B	-2	0	1	-2
C	∞	∞	0	5
D	∞	4	∞	0

π1	A	B	C	D
A	-	-	A	-
B	B	-	A	A
C	-	-	-	C
D	-	D	-	-

2nd iteration

- Find the values….



d1	A	B	C	D
A	0	∞	3	0
B	-2	0	1	-2
C	∞	∞	0	5
D	∞	4	∞	0

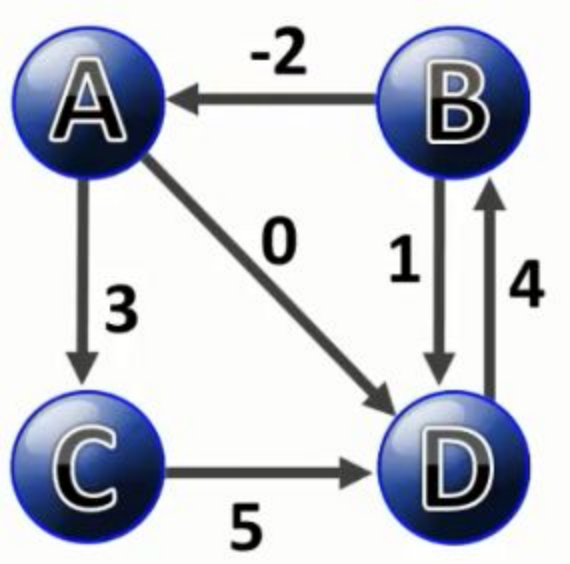
π1	A	B	C	D
A	-	-	A	-
B	B	-	A	A
C	-	-	-	C
D	-	D	-	-

d2	A	B	C	D
A	0	∞	3	0
B	-2	0	1	-2
C	∞	∞	0	5
D	2	4	5	0

π2	A	B	C	D
A	-	-	A	-
B	B	-	A	A
C	-	-	-	C
D	B	D	B	-

3rd iteration

- Find the values….



d2	A	B	C	D
A	0	∞	3	0
B	-2	0	1	-2
C	∞	∞	0	5
D	2	4	5	0

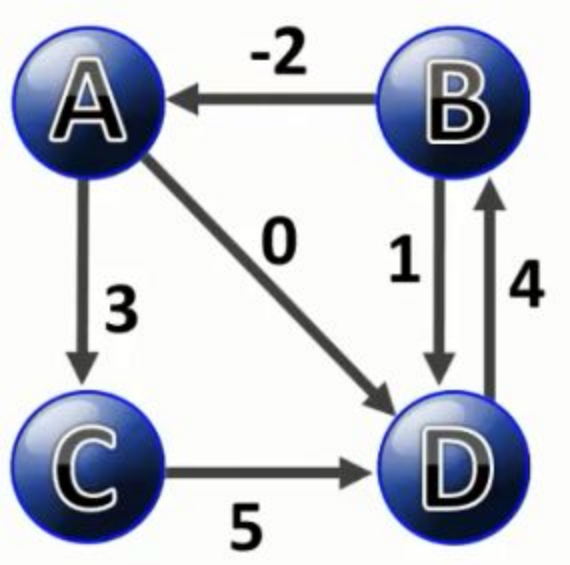
π2	A	B	C	D
A	-	-	A	-
B	B	-	A	A
C	-	-	-	C
D	B	D	B	-

d3	A	B	C	D
A	0	∞	3	0
B	-2	0	1	-2
C	∞	∞	0	5
D	2	4	5	0

π3	A	B	C	D
A	-	-	A	-
B	B	-	A	A
C	-	-	-	C
D	B	D	B	-

4^{th} iteration

- Find the values….



d3	A	B	C	D
A	0	∞	3	0
B	-2	0	1	-2
C	∞	∞	0	5
D	2	4	5	0

π3	A	B	C	D
A	-	-	A	-
B	B	-	A	A
C	-	-	-	C
D	B	D	B	-

d4	A	B	C	D
A	0	**4**	3	0
B	-2	0	1	-2
C	**7**	**9**	0	5
D	2	4	5	0

π4	A	B	C	D
A	-	**D**	A	-
B	B	-	A	A
C	**D**	**D**	-	C
D	B	D	B	-

Algorithm

```
Floyd-Warshall(W) {
        n = rows[W]
        for i=1 to n do
                for j = 1 to n do
                        D[i,j] = W[i,j]

        for k=1 to n do
                for i=1 to n do
                        for j=1 to n do
                        if (D[i,k]+D[k,j] < D[i,j])
                          D[i,j]=D[i,k}+D[k,j]

    return D
}
```

Homework: How to extract the path?

- How should we modify the algorithm to store additional information which then can be used to extract the path?